

Rounding with Guard Digits

Add $2.56_{\text{ten}} \times 10^0$ to $2.34_{\text{ten}} \times 10^2$, assuming that we have three significant decimal digits. Round to the nearest decimal number with three significant decimal digits, first with guard and round digits, and then without them.

First we must shift the smaller number to the right to align the exponents, so $2.56_{\text{ten}} \times 10^0$ becomes $0.0256_{\text{ten}} \times 10^2$. Since we have guard and round digits, we are able to represent the two least significant digits when we align exponents. The guard digit holds 5 and the round digit holds 6. The sum is

$$\begin{array}{r} 2.3400_{\text{ten}} \\ + 0.0256_{\text{ten}} \\ \hline 2.3656_{\text{ten}} \end{array}$$

Thus the sum is $2.3656_{\text{ten}} \times 10^2$. Since we have two digits to round, we want values 0 to 49 to round down and 51 to 99 to round up, with 50 being the tiebreaker. Rounding the sum up with three significant digits yields $2.37_{\text{ten}} \times 10^2$.

Doing this *without* guard and round digits drops two digits from the calculation. The new sum is then

$$\begin{array}{r} 2.34_{\text{ten}} \\ + 0.02_{\text{ten}} \\ \hline 2.36_{\text{ten}} \end{array}$$

The answer is $2.36_{\text{ten}} \times 10^2$, off by 1 in the last digit from the sum above.

Since the worst case for rounding would be when the actual number is halfway between two floating-point representations, accuracy in floating point is normally measured in terms of the number of bits in error in the least significant bits of the significand; the measure is called the number of **units in the last place**, or **ulp**. If a number were off by 2 in the least significant bits, it would be called off by 2 ulps. Provided there are no overflow, underflow, or invalid operation exceptions, IEEE 754 guarantees that the computer uses the number that is within one-half ulp.

units in the last place (ulp) The number of bits in error in the least significant bits of the significand between the actual number and the number that can be represented.

Elaboration: Although the example above really needed just one extra digit, multiply can require two. A binary product may have one leading 0 bit; hence, the normalizing step must shift the product one bit left. This shifts the guard digit into the least significant bit of the product, leaving the round bit to help accurately round the product.

IEEE 754 has four rounding modes: always round up (toward $+\infty$), always round down (toward $-\infty$), truncate, and round to nearest even. The final mode determines what to do if the number is exactly halfway in between. The U.S. *Internal Revenue Service* (IRS) always rounds 0.50 dollars up, possibly to the benefit of the IRS. A more equitable way would be to round up this case half the time and round down the other half. IEEE 754

EXAMPLE

ANSWER

sticky bit A bit used in rounding in addition to guard and round that is set whenever there are nonzero bits to the right of the round bit.

fused multiply add A floating-point instruction that performs both a multiply and an add, but rounds only once after the add.

says that if the least significant bit retained in a halfway case would be odd, add one; if it's even, truncate. This method always creates a 0 in the least significant bit in the tie-breaking case, giving the rounding mode its name. This mode is the most commonly used, and the only one that Java supports.

The goal of the extra rounding bits is to allow the computer to get the same results as if the intermediate results were calculated to infinite precision and then rounded. To support this goal and round to the nearest even, the standard has a third bit in addition to guard and round; it is set whenever there are nonzero bits to the right of the round bit. This **sticky bit** allows the computer to see the difference between $0.50 \dots 00_{\text{ten}}$ and $0.50 \dots 01_{\text{ten}}$ when rounding.

The sticky bit may be set, for example, during addition, when the smaller number is shifted to the right. Suppose we added $5.01_{\text{ten}} \times 10^{-1}$ to $2.34_{\text{ten}} \times 10^2$ in the example above. Even with guard and round, we would be adding 0.0050 to 2.34 , with a sum of 2.3450 . The sticky bit would be set, since there are nonzero bits to the right. Without the sticky bit to remember whether any 1s were shifted off, we would assume the number is equal to $2.345000 \dots 00$ and round to the nearest even of 2.34 . With the sticky bit to remember that the number is larger than $2.345000 \dots 00$, we round instead to 2.35 .

Elaboration: MIPS-64, PowerPC, SPARC64, AMD SSE5, and Intel AVX architectures provide a single instruction that does a multiply and add on three registers: $a = a + (b \times c)$. Obviously, this instruction allows potentially higher floating-point performance for this common operation. Equally important is that instead of performing two roundings—after the multiply and then after the add—which would happen with separate instructions, the multiply add instruction can perform a single rounding after the add. A single rounding step increases the precision of multiply add. Such operations with a single rounding are called **fused multiply add**. It was added to the revised IEEE 754-2008 standard (see [Section 3.12](#)). Thus, the full ARMv8 instruction offers fused multiply-adds as well (see Section 3.8).

Summary

The *Big Picture* that follows reinforces the stored-program concept from [Chapter 2](#); the meaning of the information cannot be determined just by looking at the bits, for the same bits can represent a variety of objects. This section shows that computer arithmetic is finite and thus can disagree with natural arithmetic. For example, the IEEE 754 standard floating-point representation

$$(-1)^5 \times (1 + \text{Fraction}) \times 2^{(\text{Exponent} - \text{Bias})}$$

is almost always an approximation of the real number. Computer systems must take care to minimize this gap between computer arithmetic and arithmetic in the real world, and programmers at times need to be aware of the implications of this approximation.

**The BIG
Picture**

Bit patterns have no inherent meaning. They may represent signed integers, unsigned integers, floating-point numbers, instructions, character strings, and so on. What is represented depends on the instruction that operates on the bits in the word.

The major difference between computer numbers and numbers in the real world is that computer numbers have limited size and hence limited precision; it's possible to calculate a number too big or too small to be represented in a computer word. Programmers must remember these limits and write programs accordingly.

C type	Java type	Data transfers	Operations
long long int	int	LDUR, STUR, MOVZ, MOVK	ADD, SUB, ADDI, SUBI, ADDS, SUBS, ADDIS, SUBIS, MUL, SMULH, SDIV, AND, ANDI, ORR, ORRI, EOR, EORI
unsigned long long int	—	LDUR, STUR, MOVZ, MOVK	ADD, SUB, ADDI, SUBI, ADDS, SUBS, ADDIS, SUBIS, MUL, UMULH, UDIV, AND, ANDI, ORR, ORRI, EOR, EORI
char	—	LDURB, STURB, MOVZ, MOVK	ADD, SUB, ADDI, SUBI, ADDS, SUBS, ADDIS, SUBIS, MUL, SMULH, SDIV, AND, ANDI, ORR, ORRI, EOR, EORI
—	char	LDURH, STURH, MOVZ, MOVK	ADD, SUB, ADDI, SUBI, ADDS, SUBS, ADDIS, SUBIS, MUL, UMULH, UDIV, AND, ANDI, ORR, ORRI, EOR, EORI
float	float	LDURS, STURS	FADDS, FSUBS, FMULS, FDIVS, FCMPS
double	double	LDURD, STURD	FADD, FDUBD, FMULD, FDIVD, FCMPD

In the last chapter, we presented the storage classes of the programming language C (see the *Hardware/Software Interface* section in [Section 2.7](#)). The table above shows some of the C and Java data types, the data transfer instructions, and instructions that operate on those types that appear in [Chapter 2](#) and this chapter. Note that Java omits unsigned integers.

Hardware/ Software Interface

The revised IEEE 754-2008 standard added a 16-bit floating-point format with five exponent bits. What do you think is the likely range of numbers it could represent?

Check Yourself

1. $1.0000\ 00 \times 2^0$ to $1.1111\ 1111\ 11 \times 2^{31}$, 0
2. $\pm 1.0000\ 0000\ 0 \times 2^{-14}$ to $\pm 1.1111\ 1111\ 1 \times 2^{15}$, ± 0 , $\pm \infty$, NaN
3. $\pm 1.0000\ 0000\ 00 \times 2^{-14}$ to $\pm 1.1111\ 1111\ 11 \times 2^{15}$, ± 0 , $\pm \infty$, NaN
4. $\pm 1.0000\ 0000\ 00 \times 2^{-15}$ to $\pm 1.1111\ 1111\ 11 \times 2^{14}$, ± 0 , $\pm \infty$, NaN

Elaboration: To accommodate comparisons that may include NaNs, the standard includes *ordered* and *unordered* as options for compares. Hence, the full ARMv8 instruction set has many flavors of compares to support NaNs. (Java does not support unordered compares.)

In an attempt to squeeze every bit of precision from a floating-point operation, the standard allows some numbers to be represented in unnormalized form. Rather than having a gap between 0 and the smallest normalized number, IEEE allows *denormalized numbers* (also known as *denorms* or *subnormals*). They have the same exponent as zero but a nonzero fraction. They allow a number to degrade in significance until it becomes 0, called *gradual underflow*. For example, the smallest positive single precision normalized number is

$$1.0000\ 0000\ 0000\ 0000\ 0000\ 0000_{\text{two}} \times 2^{-126}$$

but the smallest single precision denormalized number is

$$0.0000\ 0000\ 0000\ 0000\ 0000\ 0001_{\text{two}} \times 2^{-126}, \text{ or } 1.0_{\text{two}} \times 2^{-149}$$

For double precision, the denorm gap goes from 1.0×2^{-1022} to 1.0×2^{-1074} .

The possibility of an occasional unnormalized operand has given headaches to floating-point designers who are trying to build fast floating-point units. Hence, many computers cause an exception if an operand is denormalized, letting software complete the operation. Although software implementations are perfectly valid, their lower performance has lessened the popularity of denorms in portable floating-point software. Moreover, if programmers do not expect denorms, their programs may surprise them.

3.6

Parallelism and Computer Arithmetic: Subword Parallelism

Since every microprocessor in a phone, tablet, or laptop by definition has its own graphical display, as transistor budgets increased it was inevitable that support would be added for graphics operations.

Many graphics systems originally used 8 bits to represent each of the three primary colors plus 8 bits for a location of a pixel. The addition of speakers and microphones for teleconferencing and video games suggested support of sound as well. Audio samples need more than 8 bits of precision, but 16 bits are sufficient.

Every microprocessor has special support so that bytes and halfwords take up less space when stored in memory (see [Section 2.9](#)), but due to the infrequency of arithmetic operations on these data sizes in typical integer programs, there was little support beyond data transfers. Architects recognized that many graphics and audio applications would perform the same operation on vectors of these data. By partitioning the carry chains within a 128-bit adder, a processor could use **parallelism** to perform simultaneous operations on short vectors of sixteen 8-bit operands, eight 16-bit operands, four 32-bit operands, or two 64-bit operands.



The cost of such partitioned adders was small yet the speedups could be large.

Given that the parallelism occurs within a wide word, the extensions are classified as *subword parallelism*. It is also classified under the more general name of *data level parallelism*. They are known as well as vector or SIMD, for single instruction, multiple data (see [Section 6.6](#)). The rising popularity of multimedia applications led to arithmetic instructions that support narrower operations that can easily compute in parallel.

For example, ARMv8 added 32 128-bit registers (V0, V1, ..., V31) and more than 500 machine-language instructions to support subword parallelism. It supports all the subword data types you can imagine:

- 8-bit, 16-bit, 32-bit, 64-bit, and 128-bit signed and unsigned integers
- 32-bit and 64-bit floating point numbers

[Figure 3.18](#) gives a quick summary of some basic ARMv8 SIMD instructions, while [Section 3.8](#) surveys the full SIMD architecture.

Type	Description	Name	Size (bits)					FP Precision	
			8	16	32	64	128	SP	DP
Add/ Subtract	Integer add	ADD	✓	✓	✓	✓	✓		
	FP add	FADD						✓	✓
	Integer subtract	SUB	✓	✓	✓	✓	✓		
	FP subtract	FSUB						✓	✓
Multiply	Unsigned integer multiply	UMUL	✓	✓	✓	✓	✓		
	Signed integer multiply	SMUL	✓	✓	✓	✓	✓		
	FP multiply	FMUL						✓	✓
Compare	Integer compare equal	CMEQ	✓	✓	✓	✓	✓		
	FP compare equal	FCMEQ						✓	✓
Min/Max	Unsigned integer minimum	UMIN	✓	✓	✓	✓	✓		
	Signed integer minimum	SMIN	✓	✓	✓	✓	✓		
	FP minimum	FMIN						✓	✓
	Unsigned integer maximum	UMAX	✓	✓	✓	✓	✓		
	Signed integer maximum	SMAX	✓	✓	✓	✓	✓		
	FP maximum	FMAX						✓	✓
Shift	Integer shift left	SHL	✓	✓	✓	✓	✓		
	Unsigned integer shift right	USHR	✓	✓	✓	✓	✓		
	Signed integer shift right	SSHR	✓	✓	✓	✓	✓		
Logical	Bitwise AND	AND	✓	✓	✓	✓	✓		
	Bitwise OR	ORR	✓	✓	✓	✓	✓		
	Bitwise exclusive OR	EOR	✓	✓	✓	✓	✓		
Data Transfer	Load register	LDR	✓	✓	✓	✓	✓	✓	✓
	Store register	STR	✓	✓	✓	✓	✓	✓	✓

FIGURE 3.18 Example of ARMv8 SIMD instructions for subword parallelism. [Figure 3.21](#) shows the full set of SIMD instructions.

Rather than have a different assembly mnemonic for each data width, the ARMv8 assembler uses different suffixes for the SIMD registers to represent different widths, which allows it to pick the proper machine language opcode. The suffixes are B (*byte*) for 8-bit operands, H (*half*) for 16-bit operands, S (*single*) for 32-bit operands, D (*double*) for 64-bit operands, and Q (*quad*) for 128-bit operands. The programmer also specifies the number of subword operations for that data width with a number before the register name. Thus, to perform 16 parallel integer adds between the 8-bit elements in registers V2 and V3 and place the sixteen 8-bit sums in the elements of V1, we write:

```
ADD V1.16B, V2.16B, V3.16B // 16 8-bit integer adds
```

To perform four parallel 32-bit floating-point additions, we write:

```
FADD V1.4S, V2.4S, V3.4S // 4 32-bit floating-point adds
```

3.7

Real Stuff: Streaming SIMD Extensions and Advanced Vector Extensions in x86

The original MMX (*MultiMedia eXtension*) and SSE (*Streaming SIMD Extension*) instructions for the x86 included similar operations to those found in ARMv8. [Chapter 2](#) notes that in 2001 Intel added 144 instructions to its architecture as part of SSE2, including double precision floating-point registers and operations. It included eight 64-bit registers that can be used for floating-point operands. AMD expanded the number to 16 registers, called XMM, as part of AMD64, which Intel relabeled EM64T for its use. [Figure 3.19](#) summarizes the SSE and SSE2 instructions.

In addition to holding a single precision or double precision number in a register, Intel allows multiple floating-point operands to be packed into a single 128-bit SSE2 register: four single precision or two double precision. Thus, the 16 floating-point registers for SSE2 are actually 128 bits wide. If the operands can be arranged in memory as 128-bit aligned data, then 128-bit data transfers can load and store multiple operands per instruction. This packed floating-point format is supported by arithmetic operations that can compute simultaneously on four singles (PS) or two doubles (PD).

In 2011, Intel doubled the width of the registers again, now called YMM, with *Advanced Vector Extensions* (AVX). Thus, a single operation can now specify eight 32-bit floating-point operations or four 64-bit floating-point operations. The legacy SSE and SSE2 instructions now operate on the lower 128 bits of the YMM registers. Thus, to go from 128-bit and 256-bit operations, you prepend the letter “v” (for vector) in front of the SSE2 assembly language operations and then use the

Data transfer	Arithmetic	Compare
MOV[AU]{SS PS SD PD} xmm, {mem xmm}	ADD(SS PS SD PD) xmm, {mem xmm}	CMP{SS PS SD PD}
	SUB(SS PS SD PD) xmm, {mem xmm}	
MOV[HL]{PS PD} xmm, {mem xmm}	MUL(SS PS SD PD) xmm, {mem xmm}	
	DIV(SS PS SD PD) xmm, {mem xmm}	
	SQRT(SS PS SD PD) {mem xmm}	
	MAX(SS PS SD PD) {mem xmm}	
	MIN(SS PS SD PD) {mem xmm}	

FIGURE 3.19 The SSE/SSE2 floating-point instructions of the x86. xmm means one operand is a 128-bit SSE2 register, and {mem|xmm} means the other operand is either in memory or it is an SSE2 register. The table uses regular expressions to show the variations of instructions. Thus, MOV[AU]{SS|PS|SD|PD} represents the eight instructions MOVASS, MOVAPS, MOVASD, MOVAPD, MOVUSS, MOVUPS, MOVUSD, and MOVUPD. We use square brackets [] to show single-letter alternatives: A means the 128-bit operand is aligned in memory; U means the 128-bit operand is unaligned in memory; H means move the high half of the 128-bit operand; and L means move the low half of the 128-bit operand. We use the curly brackets { } with a vertical bar | to show multiple letter variations of the basic operations: SS stands for *Scalar Single* precision floating point, or one 32-bit operand in a 128-bit register; PS stands for *Packed Single* precision floating point, or four 32-bit operands in a 128-bit register; SD stands for *Scalar Double* precision floating point, or one 64-bit operand in a 128-bit register; PD stands for *Packed Double* precision floating point, or two 64-bit operands in a 128-bit register.

YMM register names instead of the XMM register name. For example, the SSE2 instruction to perform two 64-bit floating-point additions

```
addpd %xmm0, %xmm4
```

becomes

```
vaddpd %ymm0, %ymm4
```

which now produces four 64-bit floating-point multiplies. Intel has announced plans to widen the AVX registers to first 512 bits and later 1024 bits in later editions of the x86 architecture.

Elaboration: AVX also added three address instructions to x86. For example, vaddpd can now specify

```
vaddpd %ymm0, %ymm1, %ymm4 // %ymm4 = %ymm0 + %ymm1
```

instead of the standard, two address version

```
addpd %xmm0, %xmm4 // %xmm4 = %xmm4 + %xmm0
```

(Unlike LEV8, the destination is on the right in x86.) Three addresses can reduce the number of registers and instructions needed for a computation.

3.8

Real Stuff: The Rest of the ARMv8 Arithmetic Instructions

Figure 2.40 lists 63 assembly-language instructions for integer multiply, integer divide, and floating-point operations in the full ARMv8 instruction set and an impressive 245 SIMD assembly language instructions.

It also shows 18 floating-point data transfer instructions in machine language but none in assembly language. Figure 3.17, however, lists four assembly language instructions for floating-point data transfer. This apparent contradiction occurs because ARMv8 assemblers can use the names of the registers along with the generic data transfer instruction names to generate the proper opcode. For example, the ARMv8 assembler turns three instructions:

```
LDUR S1, [X23,#100]
LDUR D1, [X23,#100]
LDUR X1, [X23,#100]
```

into machine language versions of the following LEGv8 instructions:

```
LDURS S1, [X23,#100]
LDURD D1, [X23,#100]
LDUR X1, [X23,#100]
```

As mentioned in an earlier elaboration, we use distinct assembly language names for different floating-point machine language instructions in this book in the belief that there will be less confusion about how the hardware works if we keep the relationship between the two levels one-to-one, but the register names alone are sufficient for an assembler to keep things straight.

Full ARMv8 Integer and Floating-point Arithmetic Instructions

Figure 3.20 shows all 63 assembly-language integer arithmetic and floating-point instructions in the full ARMv8 instruction set, with the 15 ARMv8 arithmetic core instructions highlighted in bold. Pseudoinstructions are italicized.

Like many other ARMv8 instruction categories, there is a version of the integer multiply instruction that supplies the negative of the result (*MNEG*). There also “long” versions of the four multiply instructions, where the operands are 32 bits (*long*) instead of 64 bits (*long long*). ARMv8 also has six instructions that do both an integer multiply *and* an add or subtract for either three long (32-bit) or three long long (64-bit) operands. In fact, six of the integer multiply instructions are just pseudoinstructions of the multiply-add instructions with one of the three operands being the zero register (*XZR*). These 11 new ARMv8 integer instructions join the five multiply and divide instructions in the ARMv8 arithmetic core account for 16 instructions.

Type	Mnemonic	Instruction	Type	Mnemonic	Instruction
Integer Multiply & Divide	MUL	Multiply	Integer Mul-Add	MADD	Multiply-add
	SMULH	Signed multiply high		MSUB	Multiply-subtract
	UMULH	Unsigned multiply high		SMADDL	Signed multiply-add long
	SDIV	Signed divide		SMSUBL	Signed multiply-subtract long
	UDIV	Unsigned divide		UMADDL	Unsigned multiply-add long
	<i>SMULL</i>	Signed multiply long		UMSUBL	Unsigned multiply-subtract long
	<i>UMULL</i>	Unsigned multiply long	FP Mul-Add	FMADD	Floating-point fused multiply-add
	<i>MNEG</i>	Multiply-negate		FMSUB	Floating-point fused multiply-subtract
	<i>UMNEGL</i>	Unsigned multiply-negate long		FNMADD	Floating-point negated fused multiply-add
	<i>SMNEGL</i>	Signed multiply-negate long		FNMSUB	Floating-point negated fused multiply-subtract
FP two source operands	FADDS	Floating-point add single	FP move	FMOV	Floating-point move to/from integer or FP register
	FSUBS	Floating-point subtract single		FMOVI	Floating-point move immediate
	FMULS	Floating-point multiply single		FCSEL	Floating-point conditional select
	FDIVS	Floating-point divide single	FP sel	FCSEL	Floating-point conditional select
	FADDD	Floating-point add double		FRINTA	Floating-point round to nearest with ties to odd
	FSUBD	Floating-point subtract double		FRINTI	Floating-point round using current rounding mode
	FNMUL	Floating-point scalar multiply-negate		FRINTM	Floating-point round toward -infinity
	FMULD	Floating-point multiply double		FRINTN	Floating-point round to nearest with ties to even
	FDIVD	Floating-point divide double		FRINTP	Floating-point round toward +infinity
	FCMPS	Floating-point compare single (quiet)		FRINTX	Floating-pointl exact using current rounding mode
	FCMPD	Floating-point compare double (quiet)		FRINTZ	Floating-point round toward 0
	FCMPE	Floating-point signaling compare	FP round	FCVTAS	FP convert to signed integer, rounding to nearest odd
	FCCMP	Floating-point conditional quiet compare		FCVTAU	FP convert to unsigned integer, rounding to nearest odd
	FCCMPE	Floating-point conditional signaling compare		FCVTMS	FP convert to signed integer, rounding toward -infinity
FP one operand	FABS	Floating-point scalar absolute value		FCVTMU	FP convert to unsigned integer, rounding toward -infinity
	FNEG	Floating-point scalar negate		FCVTNS	FP convert to signed integer, rounding to nearest even
	FSQRT	Floating-point scalar square root		FCVTNU	FP convert to unsigned integer, rounding to nearest even
	FMAX	Floating-point scalar maximum		FCVTPS	FP convert to signed integer, rounding toward +infinity
FP Min/Max	FMIN	Floating-point scalar minimum		FCVTPU	FP convert to unsigned integer, rounding toward +infinity
	FMAXNM	Floating-point scalar maximum number (NaN = -Inf)		FCVTZS	FP convert to signed integer, rounding toward 0
	FMINNM	Floating-point scalar minimum number (NaN = +Inf)		FCVTZU	FP convert to unsigned integer, rounding toward 0
				SCVTF	Signed integer convert to FP, current rounding mode
				UCVTF	Unsigned integer convert to FP, current rounding mode

FIGURE 3.20 Full ARMv8 assembly-language instructions for integer and floating-point arithmetic. Instruction in bold font is in the LEGv8 core. Italicized instructions are pseudoinstructions.

Like integer multiply, there is a version of floating-point multiply that produces a negative product (**FNML**). Like integer conditional compare instruction (**CCMP**), there is a conditional version of compare that (confusingly) does a comparison only if the initial condition is true (**FCCMP**). To allow programmers to check to see if an operand is a *Not-A-Number* (NaN), there are two versions of the comparison instructions: one that doesn't cause an exception whenever one of the operands is the value NaN (quiet compare) as well as one that does (signaling compare) in case the programmer wants an exception whenever a NaN is found. Again like integer multiply, there are four floating-point instructions that do multiply followed by an add or a subtract. ARMv8 has three more instructions for floating-point operations with a single operand: absolute value, negate, and square root. These 11 instructions

join the 10 other existing instructions from the ARMv8 arithmetic core to bring our running total in 37.

Minimum and maximum floating-point operations are again a bit more complicated due to NaNs. There are two instructions that trap if an operand is a NaN and two that treat NaN as an extreme number: minus infinity for maximum or plus infinity for minimum. Not only does ARMv8 have one floating-point move instruction that can copy a value between floating-point registers or between integer registers and floating-point registers, it has one that can load a floating-point constant into a register. And once again like integer conditional select (CSEL), there is a floating-point version (FCSEL). These seven instructions get us to 44 arithmetic assembly-language instructions.

The final two categories are for rounding floating-point numbers and converting between integers and floating-point numbers. To round according to the many modes of IEEE 754, ARMv8 has seven instructions. To cover all combinations of conversions of signed and integers for the different rounding modes requires 12 more instructions. These last two categories bring us to 63 arithmetic assembly-language instructions, which matches [Figure 2.40](#).

Full ARMv8 SIMD Instructions

[Figure 3.21](#) shows all 245 assembly-language SIMD instructions in the full ARMv8 instruction set. To fit these 245 instructions into a single table, we use regular expressions to represent several instructions within a single table entry. The figure caption reviews the three regular expression operators we use.

Many SIMD instructions offer three versions:

- *Wide* means the width of the elements in the destination register and the first source registers is twice the width of the elements in the *second* source register.
- *Long* means the width of the elements in the destination register is twice the width of the elements in *all* source registers.
- *Narrow* means the width of the elements in the destination register is *half* the width of the elements in all source registers.

These three options are naturally enough indicated by the three suffixes W, L, and N. Like the non-SIMD instructions, *pair* means do the operation to a pair of SIMD registers. These instructions use the suffix P.

When dealing with narrow operations, the instruction could operate on either the lower half or the upper half of the 128-bit SIMD register containing the narrower elements. The default is use the lower half, with the suffix 2 indicating the operation is working on the upper half of the SIMD register.

The *prefixes* U, S, and F refer to the data types of unsigned integers, signed integers, and floating-point. When the elements are just plain bits, there is no prefix. There are also three instructions that use the type polynomial, and they use P as the prefix.

Most SIMD categories are self-explanatory, so we'll explain just the ones that may not be obvious. Instead of setting condition codes, SIMD compare vector sets the destination vector element to all 1s if the condition holds, or to 0 if not. While ARMv8

Type	Description	Name	Type	Description	Name
Add / Subtract	Vector add	F?ADD	Saturating Arithmetic	Integer saturating vector add	[US]QADD
	Integer vector add returning high, narrow	ADDHN2?		Integer saturating vector subtract	[US]QSUB
	Integer vector add long	[US]ADDL2?		Signed integer saturating vector accumulate of unsigned value	SQADD
	Integer vector add wide	[US]ADDW2?		Unsigned integer saturating vector accumulate of unsigned value	UQADD
	Vector add pair	FADDP		Signed integer saturating vector absolute	SQABS
	Integer vector add long pair	[US]ADDLP		Signed integer saturating vector doubling multiply-add long	SQDMLAL2?
	Integer vector add and accumulate long pair	[US]ADALP		Signed integer saturating vector doubling multiply-subtract long	SQDMLS2?
	Vector subtract	F?SUB		Signed integer saturating vector doubling multiply high half	SQDMULH
	Integer vector subtract returning high, narrow	SUBHN2?		Signed integer saturating vector doubling multiply long	SQDMULL2?
	Integer vector subtract long	[US]SUBL2?		Integer saturating vector narrow	[US]QXTN2?
Multiply / Divide / Square Root	Integer vector subtract wide	[US]SUBW2?	Multiply-Add	Signed integer saturating vector and unsigned narrow	SQXTUN2?
	Vector negate	F?NEG		Signed integer saturating vector negate	SQNEG
	Vector multiply	[FP]MUL		Signed integer vector saturating rounding doubling multiply high half	SQDMULH
	FP vector multiply extended (0xINF→2)	FMULX		Vector chained multiply-add	MLA
	Vector multiply long	[USP]MULL2?		Vector fused multiply-add	FMLA
	Vector FP divide	FDIV		Integer vector multiply-add long	[US]MLAL2?
	FP vector square root	FSQRT		Vector chained multiply-subtract	MLS
	FP reciprocal square root	FRSQRTS		Vector fused multiply-subtract	FMLS
	Vector reciprocal square root estimate	[UF]RSORTE		Integer vector multiply-subtract long	[US]MLSL2?
	Vector reciprocal estimate	[UF]RECPE	Reduction	Integer sum elements in vector	ADDV
Compare	FP vector reciprocal step	FRECPS		Integer sum elements in vector long	[US]ADDLV
	FP vector reciprocal exponent	FRECPX		Maximum element in vector	[USF]MAXV
	FP vector absolute compare greater than or equal	FACGE		FP maxNum element in vector	FMAXNMV
	FP vector absolute compare greater than	FACGT		Minimum element in vector	[USF]FMINV
	FP vector absolute compare less than or equal	FACLE		FP minNum element in vector	FMINNMV
	FP vector absolute compare less than	FACLT	Saturating Shifts	Integer saturating vector rounding shift left	[US]QORSHL
	Vector compare equal	F?CMEQ		Integer saturating vector shift right rounded narrow	[US]QORSHRN
	Vector compare greater than or equal	[CMHS] F?CMGE		Signed integer saturating vector shift right rounded unsigned narrow	SQRSHRUN
	Vector compare greater than	[CMHI] F?CMGT		Integer saturating vector shift left	[US]QSHL
	Vector compare less than or equal	[CMLS] F?CMLE		Signed integer saturating vector shift left unsigned	SQSHLU
	Vector compare less than	[CMLQ] F?CMLT		Integer saturating vector shift right narrow	[US]QSHRN
	Vector test bits	CMTST		Signed integer saturating vector shift right unsigned narrow	SQSHRUN
Logical	Bitwise vector AND	AND	Min / Max	Vector maximum	[USF]MAX
	Bitwise vector bit clear	BIC		FP vector maxNum	FMAXNM
	Bitwise vector OR	ORR		Vector minimum	[USF]MIN
	Bitwise vector OR NOT	ORN		FP vector minNum	FMINNM
	Bitwise vector exclusive OR	EOR		Vector max pair	[USF]MAXP
	Bitwise vector NOT	MVN		FP vector maxNum pair	FMAXNMP
Shifts	Integer vector shift left	SHL	Convert	Vector min pair	[USF]MINP
	Integer vector shift left long	[US]SHLL		FP vector minNum pair	FMINNMP
	Integer vector shift right	[US]SHR		Vector FP convert to (signed unsigned) integer (round to {0 x})	FCVT{Zx}[SU]
	Integer vector shift right narrow	SHRN2?		Vector integer convert to FP	[US]CVTF
	Integer vector shift left and insert	SLI		Vector convert FP precision	FCVT{NL}
	Integer vector shift right and accumulate	[US]SRA		Vector convert double to single-precision (rounding to odd)	FCVTXN
Absolute value / difference	Integer vector shift right and insert	SRI	Round	Vector FP round to integral FP value (towards x)	FRINTx
	Integer rounding vector shift left	[US]RSHL		Integer rounding vector shift right and accumulate	[US]RSRA
	Integer rounding vector shift right	[US]RSHR		Integer rounding vector subtract returning high, narrow	RSUBHN2
	Integer rounding vector shift right narrow	RSHRN2?		Integer rounding vector halving add	[US]RHADD
	Integer vector absolute difference and accumulate	[US]ABA		Integer vector rounding add returning high, narrow	RADDHN
	Integer vector absolute difference and accumulate long	[US]ABAL2?	Insert / Extract / Reverse / Duplicate	Bitwise vector select	BSL
	Vector absolute difference	[USF]ABD		Bitwise vector extract	EXT
	Integer vector absolute difference long	[USF]ABDL2?		Bitwise vector insert if (true false)	BI{TF}
	Vector absolute value	F?ABS		Vector reverse bits in bytes	RBIT
Data Transfer	Vector load {pair register}	LD{PR}		Vector reverse elements	REV{16 32 64}
	Vector store {pair register}	ST{PR}		Duplicate single vector element to all elements	DUP
	Vector move	[USF]MOV		Insert single element in another element	INS
	Vector structure/element load	LD{1234}	Vector Length	Vector element transpose	TRN{12}
	Vector replicated element load	LD{1234}R		Vector element zip	ZIP2?
	Vector structure store	ST{1234}		Vector element unzip	UZP{12}
Count Bits	Integer vector count leading {sign zero} bits	CL{SZ}		Integer vector lengthen	[US]XTL2?
	Vector count non-zero bits	CNT		Integer vector narrow	XTN
Table Lookup	Vector table lookup	TBL		Integer vector halving add	[US]HADD
	Vector table extension	TBX		Integer vector halving subtract	[US]HSUB

FIGURE 3.21 Full ARMv8 assembly language for SIMD instructions, which uses the term vector to distinguish them from single operands (scalar). To fit all 245 assembly language instructions into this small space, we use regular expressions to show the valid combinations. Question mark means 0 or 1 copies of the letter before it, so F?ADD represents the two instructions ADD and FADD. Square brackets mean there is a version for each letter in the brackets, so [US]QADD represents the two instructions UQADD and SQADD. Finally, curly brackets and a vertical line to separate the options show how to form multiple letter versions of the instructions. For example, REV{16|32|64} stands for the three instructions REV16, REV32, and REV64.

has only half of the comparisons (HS, GE, HI, GT), the programmer gets the others (LS, LE, LO, LT) by reversing the operands and using the complementary comparison. That is, $A < B$ is the same as $B \geq A$. Reductions do the operations across the elements *within* a *single* SIMD register rather than *between* elements of *different* SIMD registers as is the case for the rest of the SIMD instructions. As the category name suggests, these instructions perform the classic reduction operations of sum, minimum, and maximum. Finally, the table lookup instructions use one to four SIMD registers to act as a table. The elements of the other source register hold the indexes of the table and then the results of the parallel table lookups are stored in the elements of the destination register.

With the information above describing the terms wide, long, narrow, pair, and so on, the descriptions of the instructions within the categories are usually understandable. Here are a few that might not be:

- The elaboration in Section 3.5 explains how fused multiply-add only does one rounding instead of two as you might expect from doing two operations in one instruction. As is often the case, rather than choosing between them, ARMv8 offers both *fused* multiply-adds (one rounding) and *chained* multiply-adds (two roundings).
- Integer vector shift left and insert (SLI) provides a way to combine bits from two vectors.
- Integer vector shift right and accumulate instructions (SSRA, USRA) are useful when intermediate calculations are made at a higher precision before the result is added to a lower precision accumulator.
- Vector structure/element load instructions (LD1, LD2, LD3, LD4) loads structures of one, two, three, or four elements into SIMD registers.

3.9

Going Faster: Subword Parallelism and Matrix Multiply

To demonstrate the performance impact of subword parallelism, we'll run the same code on the Intel Core i7 first without AVX and then with it. [Figure 3.22](#) shows an unoptimized version of a matrix-matrix multiply written in C. As we saw in Section 3.5, this program is commonly called *DGEMM*, which stands for Double precision GEneral Matrix Multiply. Starting with this edition, we have added a new section entitled "Going Faster" to demonstrate the performance benefit of adapting software to the underlying hardware, in this case the Sandy Bridge version of the Intel Core i7 microprocessor. This new section in Chapters 3, 4, 5, and 6 will incrementally improve DGEMM performance using the ideas that each chapter introduces.

[Figure 3.23](#) shows the x86 assembly language output for the inner loop of [Figure 3.22](#). The five floating point-instructions start with a `v` like the AVX instructions, but note that they use the XMM registers instead of YMM, and they include `sd`

```

1. void dgemm (size_t n, double* A, double* B, double* C)
2. {
3.     for (size_t i = 0; i < n; ++i)
4.         for (size_t j = 0; j < n; ++j)
5.             {
6.                 double cij = C[i+j*n]; /* cij = C[i][j] */
7.                 for(size_t k = 0; k < n; k++ )
8.                     cij += A[i+k*n] * B[k+j*n]; /*cij+=A[i][k]*B[k][j]*/
9.                 C[i+j*n] = cij; /* C[i][j] = cij */
10.            }
11. }

```

FIGURE 3.22 Unoptimized C version of a double precision matrix multiply, widely known as DGEMM for Double-precision GEneral Matrix Multiply. Because we are passing the matrix dimension as the parameter n , this version of DGEMM uses single-dimensional versions of matrices C , A , and B and address arithmetic to get better performance instead of using the more intuitive two-dimensional arrays that we saw in Section 3.5. The comments remind us of this more intuitive notation.

```

1. vmovsd (%r10),%xmm0           // Load 1 element of C into %xmm0
2. mov     %rsi,%rcx              // register %rcx = %rsi
3. xor     %eax,%eax              // register %eax = 0
4. vmovsd (%rcx),%xmm1           // Load 1 element of B into %xmm1
5. add     %r9,%rcx               // register %rcx = %rcx + %r9
6. vmulsd (%r8,%rax,8),%xmm1,%xmm1 // Multiply %xmm1,element of A
7. add     $0x1,%rax              // register %rax = %rax + 1
8. cmp     %eax,%edi              // compare %eax to %edi
9. vaddsd %xmm1,%xmm0,%xmm0       // Add %xmm1, %xmm0
10. jg      30 <dgemm+0x30>        // jump if %eax > %edi
11. add     $0x1,%r11              // register %r11 = %r11 + 1
12. vmovsd %xmm0,(%r10)           // Store %xmm0 into C element

```

FIGURE 3.23 The x86 assembly language for the body of the nested loops generated by compiling the unoptimized C code in Figure 3.22. Although it is dealing with just 64 bits of data, the compiler uses the AVX version of the instructions instead of SSE2 presumably so that it can use three address per instruction instead of two (see the *Elaboration* in Section 3.7).

```

1. //include <x86intrin.h>
2. void dgemm (size_t n, double* A, double* B, double* C)
3. {
4.     for ( size_t i = 0; i < n; i+=4 )
5.         for ( size_t j = 0; j < n; j++ ) {
6.             __m256d c0 = __mm256_load_pd(C+i+j*n); /* c0 = C[i][j] */
7.             for( size_t k = 0; k < n; k++ )
8.                 c0 = _mm256_add_pd(c0, /* c0 += A[i][k]*B[k][j] */
9.                                     _mm256_mul_pd(_mm256_load_pd(A+i+k*n),
10.                                                    _mm256_broadcast_sd(B+k+j*n)));
11.             _mm256_store_pd(C+i+j*n, c0); /* C[i][j] = c0 */
12.         }
13. }

```

FIGURE 3.24 Optimized C version of DGEMM using C intrinsics to generate the AVX subword-parallel instructions for the x86. Figure 3.25 shows the assembly language produced by the compiler for the inner loop.

in the name, which stands for scalar double precision. We'll define the subword parallel instructions shortly.

While compiler writers may eventually be able to produce high-quality code routinely that uses the AVX instructions of the x86, for now we must “cheat” by using C intrinsics that more or less tell the compiler exactly how to produce good code. Figure 3.24 shows the enhanced version of Figure 3.22 for which the Gnu C compiler produces AVX code. Figure 3.25 shows annotated x86 code that is the output of compiling using gcc with the -O3 level of optimization.

The declaration on line 6 of Figure 3.24 uses the `__m256d` data type, which tells the compiler the variable will hold four double-precision floating-point values. The intrinsic `__mm256_load_pd()` also on line 6 uses AVX instructions to load four double-precision floating-point numbers in parallel (`_pd`) from the matrix C into `c0`. The address calculation `C+i+j*n` on line 6 represents element `C[i+j*n]`. Symmetrically, the final step on line 11 uses the intrinsic `__mm256_store_pd()` to store four double-precision floating-point numbers from `c0` into the matrix C. As we're going through four elements each iteration, the outer *for* loop on line 4 increments *i* by 4 instead of by 1 as on line 3 of Figure 3.22.

Inside the loops, on line 9 we first load four elements of A again using `__mm256_load_pd()`. To multiply these elements by one element of B, on line 10 we first use the intrinsic `__mm256_broadcast_sd()`, which makes four identical copies of the scalar double precision number—in this case an element of B—in one of the YMM

```

1. vmovapd (%r11),%ymm0           // Load 4 elements of C into %ymm0
2. mov     %rbx,%rcx              // register %rcx = %rbx
3. xor     %eax,%eax              // register %eax = 0
4. vbroadcastsd (%rax,%r8,1),%ymm1 // Make 4 copies of B element
5. add     $0x8,%rax              // register %rax = %rax + 8
6. vmulpd (%rcx),%ymm1,%ymm1      // Parallel mul %ymm1,4 A elements
7. add     %r9,%rcx              // register %rcx = %rcx + %r9
8. cmp     %r10,%rax             // compare %r10 to %rax
9. vaddpd %ymm1,%ymm0,%ymm0       // Parallel add %ymm1, %ymm0
10. jne     50 <dgemm+0x50>        // jump if not %r10 != %rax
11. add     $0x1,%esi            // register %esi = %esi + 1
12. vmovapd %ymm0,(%r11)          // Store %ymm0 into 4 C elements

```

FIGURE 3.25 The x86 assembly language for the body of the nested loops generated by compiling the optimized C code in Figure 3.24. Note the similarities to Figure 3.23, with the primary difference being that the five floating-point operations are now using YMM registers and using the pd versions of the instructions for parallel double precision instead of the sd version for scalar double precision.

registers. We then use `_mm256_mul_pd()` on line 9 to multiply the four double-precision results in parallel. Finally, `_mm256_add_pd()` on line 8 adds the four products to the four sums in `c0`.

Figure 3.25 shows resulting x86 code for the body of the inner loops produced by the compiler. You can see the five AVX instructions—they all start with `v` and four of the five use `pd` for parallel double precision—that correspond to the C intrinsics mentioned above. The code is very similar to that in Figure 3.23 above: both use 12 instructions, the integer instructions are nearly identical (but different registers), and the floating-point instruction differences are generally just going from *scalar double* (`sd`) using XMM registers to *parallel double* (`pd`) with YMM registers. The one exception is line 4 of Figure 3.25. Every element of *A* must be multiplied by one element of *B*. One solution is to place four identical copies of the 64-bit *B* element side-by-side into the 256-bit YMM register, which is just what the instruction `vbroadcastsd` does.

For matrices of dimensions of 32 by 32, the unoptimized DGEMM in Figure 3.22 runs at 1.7 GigaFLOPS (FLOating point Operations Per Second) on one core of a 2.6 GHz Intel Core i7 (Sandy Bridge). The optimized code in Figure 3.24 performs at 6.4 GigaFLOPS. The AVX version is 3.85 times as fast, which is very close to the factor of 4.0 increase that you might hope for from performing four times as many operations at a time by using **subword parallelism**.



PARALLELISM

Elaboration: As mentioned in the *Elaboration* in [Section 1.6](#), Intel offers Turbo mode that temporarily runs at a higher clock rate until the chip gets too hot. This Intel Core i7 (Sandy Bridge) can increase from 2.6 GHz to 3.3 GHz in Turbo mode. The results above are with Turbo mode turned off. If we turn it on, we improve all the results by the increase in the clock rate of $3.3/2.6 = 1.27$ to 2.1 GFLOPS for unoptimized DGEMM and 8.1 GFLOPS with AVX. Turbo mode works particularly well when using only a single core of an eight-core chip, as in this case, as it lets that single core use much more than its fair share of power since the other cores are idle.

3.10 Fallacies and Pitfalls

Thus mathematics may be defined as the subject in which we never know what we are talking about, nor whether what we are saying is true.

Bertrand Russell, *Recent Words on the Principles of Mathematics*, 1901

Arithmetic fallacies and pitfalls generally stem from the difference between the limited precision of computer arithmetic and the unlimited precision of natural arithmetic.

Fallacy: Just as a left shift instruction can replace an integer multiply by a power of 2, a right shift is the same as an integer division by a power of 2.

Recall that a binary number x , where x_i means the i th bit, represents the number

$$\dots + (x^3 \times 2^3) + (x^2 \times 2^2) + (x^1 \times 2^1) + (x^0 \times 2^0)$$

Shifting the bits of c right by n bits would seem to be the same as dividing by 2^n . And this *is* true for unsigned integers. The problem is with signed integers. For example, suppose we want to divide -5_{ten} by 4_{ten} ; the quotient should be -1_{ten} . The two's complement representation of -5_{ten} is

```
11111111 11111111 11111111 11111111 11111111 11111111 11111111 11111011two
```

According to this fallacy, shifting right by two should divide by 4_{ten} (2^2):

```
00111111 11111111 11111111 11111111 11111111 11111111 11111111 11111110two
```

With a 0 in the sign bit, this result is clearly wrong. The value created by the shift right is actually $4,611,686,018,427,387,902_{\text{ten}}$ instead of -1_{ten} .

A solution would be to have an arithmetic right shift that extends the sign bit instead of shifting in 0s. A 2-bit arithmetic shift right of -5_{ten} produces

```
11111111 11111111 11111111 11111111 11111111 11111111 11111111 11111110two
```

The result is -2_{ten} instead of -1_{ten} ; close, but no cigar.

Pitfall: Floating-point addition is not associative.

Associativity holds for a sequence of two's complement integer additions, even if the computation overflows. Alas, because floating-point numbers are approximations of real numbers and because computer arithmetic has limited precision, it does not hold for floating-point numbers. Given the great range of numbers that can be represented in floating point, problems occur when adding two large numbers of opposite signs plus a small number. For example, let's see if $c + (a + b) = (c + a) + b$. Assume $c = -1.5_{\text{ten}} \times 10^{38}$, $a = 1.5_{\text{ten}} \times 10^{38}$, and $b = 1.0$, and that these are all single precision numbers.

$$\begin{aligned} c + (a + b) &= -1.5_{\text{ten}} \times 10^{38} + (1.5_{\text{ten}} \times 10^{38} + 1.0) \\ &= -1.5_{\text{ten}} \times 10^{38} + (1.5_{\text{ten}} \times 10^{38}) \\ &= 0.0 \\ c + (a + b) &= (-1.5_{\text{ten}} \times 10^{38} + 1.5_{\text{ten}} \times 10^{38}) + 1.0 \\ &= (0.0_{\text{ten}}) + 1.0 \\ &= 1.0 \end{aligned}$$

Since floating-point numbers have limited precision and result in approximations of real results, $1.5_{\text{ten}} \times 10^{38}$ is so much larger than 1.0_{ten} that $1.5_{\text{ten}} \times 10^{38} + 1.0$ is still $1.5_{\text{ten}} \times 10^{38}$. That is why the sum of c , a , and b is 0.0 or 1.0 , depending on the order of the floating-point additions, so $c + (a + b) \neq (c + a) + b$. Therefore, floating-point addition is *not* associative.

Fallacy: Parallel execution strategies that work for integer data types also work for floating-point data types.

Programs have typically been written first to run sequentially before being rewritten to run concurrently, so a natural question is, "Do the two versions get the same answer?" If the answer is no, you presume there is a bug in the parallel version that you need to track down.

This approach assumes that computer arithmetic does not affect the results when going from sequential to parallel. That is, if you were to add a million numbers together, you would get the same results whether you used one processor or 1000 processors. This assumption holds for two's complement integers, since integer addition is associative. Alas, since floating-point addition is not associative, the assumption does not hold.

A more vexing version of this fallacy occurs on a parallel computer where the operating system scheduler may use a different number of processors depending on what other programs are running on a parallel computer. As the varying number of processors from each run would cause the floating-point sums to be calculated in different orders, getting slightly different answers each time despite running identical code with identical input may flummox unaware parallel programmers.

Given this quandary, programmers who write parallel code with floating-point numbers need to verify whether the results are credible, even if they don't give the exact same answer as the sequential code. The field that deals with such issues is called numerical analysis, which is the subject of textbooks in its own right. Such concerns are

one reason for the popularity of numerical libraries such as LAPACK and SCALAPACK, which have been validated in both their sequential and parallel forms.

Fallacy: Only theoretical mathematicians care about floating-point accuracy.

Newspaper headlines of November 1994 prove this statement is a fallacy (see Figure 3.26). The following is the inside story behind the headlines.

The Pentium uses a standard floating-point divide algorithm that generates multiple quotient bits per step, using the most significant bits of divisor and dividend to guess the next 2 bits of the quotient. The guess is taken from a lookup table containing -2 , -1 , 0 , $+1$, or $+2$. The guess is multiplied by the divisor and subtracted from the remainder to generate a new remainder. Like nonrestoring division, if a previous guess gets too large a remainder, the partial remainder is adjusted in a subsequent pass.

Evidently, there were five elements of the table from the 80486 that Intel engineers thought could never be accessed, and they optimized the PLA to return 0 instead of 2 in these situations on the Pentium. Intel was wrong; while the first 11 bits were always correct, errors would show up occasionally in bits 12 to 52, or the 4th to 15th decimal digits.



FIGURE 3.26 A sampling of newspaper and magazine articles from November 1994, including the *New York Times*, *San Jose Mercury News*, *San Francisco Chronicle*, and *Infoworld*. The Pentium floating-point divide bug even made the “Top 10 List” of the *David Letterman Late Show* on television. Intel eventually took a \$300 million write-off to replace the buggy chips.

A math professor at Lynchburg College in Virginia, Thomas Nicely, discovered the bug in September 1994. After calling Intel technical support and getting no official reaction, he posted his discovery on the Internet. This post led to a story in a trade magazine, which in turn caused Intel to issue a press release. It called the bug a glitch that would affect only theoretical mathematicians, with the average spreadsheet user seeing an error every 27,000 years. IBM Research soon counterclaimed that the average spreadsheet user would see an error every 24 days. Intel soon threw in the towel by making the following announcement on December 21:

We at Intel wish to sincerely apologize for our handling of the recently publicized Pentium processor flaw. The Intel Inside symbol means that your computer has a microprocessor second to none in quality and performance. Thousands of Intel employees work very hard to ensure that this is true. But no microprocessor is ever perfect. What Intel continues to believe is technically an extremely minor problem has taken on a life of its own. Although Intel firmly stands behind the quality of the current version of the Pentium processor, we recognize that many users have concerns. We want to resolve these concerns. Intel will exchange the current version of the Pentium processor for an updated version, in which this floating-point divide flaw is corrected, for any owner who requests it, free of charge anytime during the life of their computer.

Analysts estimate that this recall cost Intel \$500 million, and Intel engineers did not get a Christmas bonus that year.

This story brings up a few points for everyone to ponder. How much cheaper would it have been to fix the bug in July 1994? What was the cost to repair the damage to Intel's reputation? And what is the corporate responsibility in disclosing bugs in a product so widely used and relied upon as a microprocessor?

3.11 Concluding Remarks

Over the decades, computer arithmetic has become largely standardized, greatly enhancing the portability of programs. Two's complement binary integer arithmetic is found in every computer sold today, and if it includes floating point support, it offers the IEEE 754 binary floating-point arithmetic.

Computer arithmetic is distinguished from paper-and-pencil arithmetic by the constraints of limited precision. This limit may result in invalid operations through calculating numbers larger or smaller than the predefined limits. Such anomalies, called "overflow" or "underflow," may result in exceptions or interrupts, emergency events similar to unplanned subroutine calls. Chapters 4 and 5 discuss exceptions in more detail.



Floating-point arithmetic has the added challenge of being an approximation of real numbers, and care needs to be taken to ensure that the computer number selected is the representation closest to the actual number. The challenges of imprecision and limited representation of floating point are part of the inspiration for the field of numerical analysis. The switch to **parallelism** will shine the searchlight on numerical analysis again, as solutions that were long considered safe on sequential computers must be reconsidered when trying to find the fastest algorithm for parallel computers that still achieves a correct result.

Data-level parallelism, specifically subword parallelism, offers a simple path to higher performance for programs that are intensive in arithmetic operations for either integer or floating-point data. We showed that we could speed up matrix multiply nearly fourfold by using instructions that could execute four floating-point operations at a time.

With the explanation of computer arithmetic in this chapter comes a description of much more of the LEGv8 instruction set. One point of confusion is the instructions covered in these chapters versus instructions executed by LEGv8 chips versus the instructions accepted by LEGv8 assemblers. Two figures try to make this clear.

Figure 3.27 lists the LEGv8 instructions covered in this chapter and Chapter 2. We call the set of instructions on the left-hand side of the figure the *LEGv8 core*. The instructions on the right we call the *LEGv8 arithmetic core*.

Figure 3.28 gives the popularity of the LEGv8 instructions for SPEC CPU2006 integer and floating-point benchmarks. All instructions are listed that were responsible for at least 0.2% of the instructions executed.

Note that although programmers and compiler writers may use the full ARMv8 instruction set to have a richer menu of options, LEGv8 core instructions dominate integer SPEC CPU2006 execution, and the integer core plus arithmetic core dominate SPEC CPU2006 floating point, as the table below shows.

Instruction subset	Integer	Fl. pt.
LEGv8 core	98%	31%
LEGv8 arithmetic core	2%	66%
Remaining ARMv8	0%	3%

For the rest of the book, we concentrate on the LEGv8 core instructions—the integer instruction set excluding multiply and divide—to make the explanation of computer design easier. As you can see, the LEGv8 core includes the most popular LEGv8 instructions; be assured that understanding a computer that runs the LEGv8 core will give you sufficient background to understand even more ambitious computers. No matter what the instruction set or its size—ARMv8, ARMv7, MIPS, x86—never forget that bit patterns have no inherent meaning. The same bit pattern may represent a signed integer, unsigned integer, floating-point number, string, instruction, and so on. In stored-program computers, it is the operation on the bit pattern that determines its meaning.

LEGV8 core instructions	Name	Format	LEGV8 arithmetic core	Name	Format
add	ADD	R	multiply	MUL	R
subtract	SUB	R	signed multiply high	SMULH	R
add immediate	ADDI	I	unsigned multiply high	UMULH	R
subtract immediate	SUBI	I	signed divide	SDIV	R
add and set flags	ADDS	R	unsigned divide	UDIV	R
subtract and set flags	SUBS	R	floating-point add single	FADDS	R
add immediate and set flags	ADDIS	I	floating-point subtract single	FSUBS	R
subtract immediate and set flags	SUBIS	I	floating-point multiply single	FMULS	R
load register	LDUR	D	floating-point divide single	FDIVS	R
store register	STUR	D	floating-point add double	FADDD	R
load signed word	LDURSW	D	floating-point subtract double	FSUBD	R
store word	STURW	D	floating-point multiply double	FMULD	R
load half	LDURH	D	floating-point divide double	FDIVD	R
store half	STURH	D	floating-point compare single	FCMPS	R
load byte	LDURB	D	floating-point compare double	FCMPD	R
store byte	STURB	D	load single floating-point	LDURS	D
load exclusive register	LDXR	D	load double floating-point	LDURD	D
store exclusive register	STXR	D	store single floating-point	STURS	D
move wide with zero	MOVZ	IM	store double floating-point	STURD	D
move wide with keep	MOVK	IM			
and	AND	R			
inclusive or	ORR	R			
exclusive or	EOR	R			
and immediate	ANDI	I			
inclusive or immediate	ORRI	I			
exclusive or immediate	EORI	I			
logical shift left	LSL	R			
logical shift right	LSR	R			
compare and branch on equal 0	CBZ	CB			
compare and branch on not equal 0	CBNZ	CB			
branch conditionally	B, cond	CB			
branch	B	B			
branch to register	BR	R			
branch with link	BL	B			

FIGURE 3.27 The LEGv8 instruction set. This book concentrates on the instructions in the left column. This information is also found in columns 1 and 2 of the LEGv8 Reference Data Card at the front of this book.

Core ARMv8	Name	Integer	Fl. pt.	Arithmetic core + ARMv8	Name	Integer	Fl. pt.
add	ADD	5.2%	3.5%	FP add double	FADDD	0.0%	10.6%
subtract	SUB	2.2%	0.6%	FP subtract double	FSUBD	0.0%	4.9%
add immediate	ADDI	6.0%	4.8%	FP multiply double	FMULD	0.0%	15.0%
subtract immediate	SUBI	3.0%	2.4%	FP divide double	FDIVD	0.0%	0.2%
add and set flags	ADDS	1.3%	0.3%	FP add single	FADDS	0.0%	1.5%
subtract and set flags	SUBS	12.0%	2.8%	FP subtract single	FSUBS	0.0%	1.8%
add immediate and set flags	ADDIS	0.4%	0.0%	FP multiply single	FMULS	0.0%	2.4%
subtract immediate and set flags	SUBIS	3.8%	0.4%	FP divide single	FDIVS	0.0%	0.2%
load register	LDUR	18.6%	5.8%	load word to FP double	LDURD	0.0%	17.5%
store register	STUR	7.6%	2.0%	store word to FP double	STURD	0.0%	4.9%
load half	LDURH	1.3%	0.0%	load word to FP single	LDURS	0.0%	4.2%
store half	STURH	0.1%	0.0%	store word to FP single	STURS	0.0%	1.1%
load byte	LDURB	3.7%	0.1%	floating-point compare double	FCMPD	0.0%	0.6%
store byte	STURB	0.6%	0.0%	multiply	MUL	0.0%	0.2%
move wide with zero	MOVZ	3.0%	0.5%	shift right arithmetic	ASR	0.5%	0.3%
move wide with keep	MOVK	0.3%	0.1%				
and	AND	0.2%	0.1%				
inclusive or	ORR	4.0%	1.2%				
exclusive or	EOR	0.4%	0.2%				
and immediate	ANDI	0.7%	0.2%				
inclusive or immediate	ORRI	1.0%	0.2%				
logical shift left	LSL	4.4%	1.9%				
logical shift right	LSR	1.1%	0.5%				
compare and branch on equal 0	CBZ	0.9%	2.1%				
compare and branch on not equal 0	CBNZ	0.9%	1.3%				
branch conditionally	B.cond	15.3%	0.6%				
branch to register	BR	1.1%	0.2%				
branch with link	BL	0.7%	0.2%				

FIGURE 3.28 The frequency of the LEGv8 instructions for SPEC CPU2006 integer and floating point. All instructions that accounted for at least 0.2% of the instructions are included in the table. Pseudoinstructions are converted into LEGv8 before execution, and hence do not appear here.

Gresham's Law ("Bad money drives out Good") for computers would say, "The Fast drives out the Slow even if the Fast is wrong."

W. Kahan, 1992



Historical Perspective and Further Reading

This section surveys the history of the floating point going back to von Neumann, including the surprisingly controversial IEEE standards effort, plus the rationale for the 80-bit stack architecture for floating point in the x86. See the rest of [Section 3.12](#) online.



Historical Perspective and Further Reading

This section surveys the history of the floating point going back to von Neumann, including the surprisingly controversial IEEE standards effort, the rationale for the 80-bit stack architecture for floating point in the IA-32, and an update on the next round of the standard.

At first it may be hard to imagine a subject of less interest than the correctness of computer arithmetic or its accuracy, and harder still to understand why a subject so old and mathematical should be so contentious. Computer arithmetic is as old as computing itself, and some of the subject's earliest notions, like the economical reuse of registers during serial multiplication and division, still command respect today. [Maurice Wilkes \[1985\]](#) recalled a conversation about that notion during his visit to the United States in 1946, before the earliest stored-program computer had been built:

... a project under von Neumann was to be set up at the Institute of Advanced Studies in Princeton.... Goldstine explained to me the principal features of the design, including the device whereby the digits of the multiplier were put into the tail of the accumulator and shifted out as the least significant part of the product was shifted in. I expressed some admiration at the way registers and shifting circuits were arranged ... and Goldstine remarked that things of that nature came very easily to von Neumann.

There is no controversy here; it can hardly arise in the context of exact integer arithmetic, so long as there is general agreement on what integer the correct result should be. However, as soon as approximate arithmetic enters the picture, so does controversy, as if one person's "negligible" must be another's "everything."

The First Dispute

Floating-point arithmetic kindled disagreement before it was ever built. John von Neumann was aware of Konrad Zuse's proposal for a computer in Germany in 1939 that was never built, probably because the floating point made it appear too complicated to finish before the Germans expected World War II to end. Hence, von Neumann refused to include it in the computer he built at Princeton. In an influential report coauthored in 1946 with H. H. Goldstine and A. W. Burks, he gave the arguments for and against floating point. In favor:

... to retain in a sum or product as many significant digits as possible and ... to free the human operator from the burden of estimating and inserting into a problem "scale factors"—multiplication constants which serve to keep numbers within the limits of the machine.

Gresham's Law ("Bad money drives out Good") for computers would say, "The Fast drives out the Slow even if the Fast is wrong."

W. Kahan, 1992

Floating point was excluded for several reasons:

There is, of course, no denying the fact that human time is consumed in arranging for the introduction of suitable scale factors. We only argue that the time consumed is a very small percentage of the total time we will spend in preparing an interesting problem for our machine. The first advantage of the floating point is, we feel, somewhat illusory. In order to have such a floating point, one must waste memory capacity which could otherwise be used for carrying more digits per word. It would therefore seem to us not at all clear whether the modest advantages of a floating binary point offset the loss of memory capacity and the increased complexity of the arithmetic and control circuits.

The argument seems to be that most bits devoted to exponent fields would be bits wasted. Experience has proven otherwise.

One software approach to accommodate reals without floating-point hardware was called *floating vectors*; the idea was to compute at runtime one scale factor for a whole array of numbers, choosing the scale factor so that the array's biggest number would barely fill its field. By 1951, James H. Wilkinson had used this scheme extensively for matrix computations. The problem proved to be that a program might encounter a very large value, and hence the scale factor must accommodate these rare sizeable numbers. The common numbers would thus have many leading 0s, since all numbers had to use a single scale factor. Accuracy was sacrificed, because the least significant bits had to be lost on the right to accommodate leading 0s. This wastage became obvious to practitioners on early computers that displayed all their memory bits as dots on cathode ray tubes (like TV screens) because the loss of precision was visible. Where floating point deserved to be used, no practical alternative existed.

Thus, true floating-point hardware became popular because it was useful. By 1957, floating-point hardware was almost ubiquitous. A decimal floating-point unit was available for the IBM 650, and soon the IBM 704, 709, 7090, 7094 ... series would offer binary floating-point hardware for double as well as single precision.

As a result, everybody had floating point, but every implementation was different.

Diversity versus Portability

Since roundoff introduces some error into almost all floating-point operations, to complain about another bit of error seems picayune. So for 20 years, nobody complained much that those operations behaved a little differently on different computers. If software required clever tricks to circumvent those idiosyncrasies and finally deliver results correct in all but the last several bits, such tricks were deemed part of the programmer's art. For a long time, matrix computations mystified most people who had no notion of error analysis; perhaps this continues to be true. That

may be why people are still surprised that numerically stable matrix computations depend upon the quality of arithmetic in so few places, far fewer than are generally supposed. Books by Wilkinson and widely used software packages like Linpack and Eispack sustained a false impression, widespread in the early 1970s, that a modicum of skill sufficed to produce *portable* numerical software.

“Portable” here means that the software is distributed as source code in some standard language to be compiled and executed on practically any commercially significant computer, and that it will then perform its task as well as any other program performs that task on that computer. Insofar as numerical software has often been thought to consist entirely of computer-independent mathematical formulas, its portability has commonly been taken for granted; the mistake in that presumption will become clear shortly.

Packages like Linpack and Eispack cost so much to develop—over a hundred dollars per line of Fortran delivered—that they could not have been developed without U.S. government subsidy; their portability was a precondition for that subsidy. But nobody thought to distinguish how various components contributed to their cost. One component was algorithmic—devise an algorithm that deserves to work on at least one computer despite its roundoff and over-/underflow limitations. Another component was the software engineering effort required to achieve and confirm portability to the diverse computers commercially significant at the time; this component grew more onerous as ever more diverse floating-point arithmetics blossomed in the 1970s. And yet scarcely anybody realized how much that diversity inflated the cost of such software packages.

A Backward Step

Early evidence that somewhat different arithmetics could engender grossly different software development costs was presented in 1964. It happened at a meeting of SHARE, the IBM mainframe users’ group, at which IBM announced System/360, the successor to the 7094 series. One of the speakers described the tricks he had been forced to devise to achieve a level of quality for the S/360 library that was not quite so high as he had previously achieved for the 7094.

Von Neumann could have foretold part of the trouble, had he still been alive. In 1948, he and Goldstine had published a lengthy error analysis so difficult and so pessimistic that hardly anybody paid attention to it. It did predict correctly, however, that computations with larger arrays of data would probably fall prey to roundoff more often. IBM S/360s had bigger memories than 7094s, so data arrays could grow larger, and they did. To make matters worse, the S/360s had narrower single precision words (32 bits versus 36) and used a cruder arithmetic (hexadecimal or base 16 versus binary or base 2) with consequently poorer worst-case precision (21 significant bits versus 27) than the old 7094s. Consequently,

software that had almost always provided (barely) satisfactory accuracy on 7094s too often produced inaccurate results when run on S/360s. The quickest way to recover adequate accuracy was to replace old codes' single precision declarations with double precision before recompilation for the S/360. This practice exercised S/360 double precision far more than had been expected.

The early S/360's worst troubles were caused by lack of a guard digit in double precision. This lack showed up in multiplication as a failure of identities like $1.0 * x = x$ because multiplying x by 1.0 dropped x 's last hexadecimal digit (4 bits). Similarly, if x and y were very close but had different exponents, subtraction dropped off the last digit of the smaller operand before computing $x - y$. This final aberration in double precision undermined a precious theorem that single precision then (and now) honored: If $1/2 \leq x/y \leq 2$, then no rounding error can occur when $x - y$ is computed; it must be computed exactly.

Innumerable computations had benefited from this minor theorem, most often unwittingly, for several decades before its first formal announcement and proof. We had been taking all this stuff for granted.

The identities and theorems about exact relationships that persisted, despite roundoff, with reasonable implementations of approximate arithmetic were not appreciated until they were lost. Previously, it had been thought that the things to matter were precision (how many significant digits were carried) and range (the spread between over-/underflow thresholds). Since the S/360's double precision had more precision and wider range than the 7094's, software was expected to continue to work at least as well as before. But it didn't.

Programmers who had matured into program managers were appalled at the cost of converting 7094 software to run on S/360s. A small subcommittee of SHARE proposed improvements to the S/360 floating point. This committee was surprised and grateful to get a fair part of what they asked for from IBM, including all-important guard digits. By 1968, these had been retrofitted to S/360s in the field at considerable expense; worse than that was customers' loss of faith in IBM's infallibility (a lesson learned by Intel 30 years later). IBM employees who can remember the incident still shudder.

The People Who Built the Bombs

Seymour Cray was associated for decades with the CDC and Cray computers that were, when he built them, the world's biggest and fastest. He always understood what his customers wanted most: *speed*. And he gave it to them even if, in so doing, he also gave them arithmetics more "interesting" than anyone else's. Among his customers have been the great government laboratories like those at Livermore and Los Alamos, where nuclear weapons were designed. The challenges of "interesting" arithmetics were pretty tame to people who had to overcome Mother Nature's challenges.

Perhaps all of us could learn to live with arithmetic idiosyncrasy if only one computer's idiosyncrasies had to be endured. Instead, when accumulating different computers' different anomalies, software dies the Death of a Thousand Cuts. Here is an example from Cray's computers:

```
if (x == 0.0)    y = 17.0 else y = z/x
```

Could this statement be stopped by a divide-by-zero error? On a CDC 6600 it could. The reason was a conflict between the 6600's adder, where x was compared with 0.0, and the multiplier and divider. The adder's comparison examined x 's leading 13 bits, which sufficed to distinguish zero from normal nonzero floating-point numbers x . The multiplier and divider examined only 12 leading bits. Consequently, tiny numbers existed that were nonzero to the adder but zero to the multiplier and divider! To avoid disasters with these tiny numbers, programmers learned to replace statements like the one above with

```
if (1.0 * x == 0.0)    y = 17.0 else y = z/x
```

But this statement is unsafe to use in would-be portable software because it malfunctions obscurely on other computers designed by Cray, the ones marketed by Cray Research, Inc. If x was so huge that $2.0 * x$ would overflow, then $1.0 * x$ might overflow too! Overflow happens because Cray computers check the product's exponent before the product's exponent has been normalized, just to save the delay of a single AND gate.

Rounding error anomalies that are far worse than the over-/underflow anomaly just discussed also affect Cray computers. The worst error came from the lack of a guard digit in add/subtract, an affliction of IBM S/360s. Further bad luck for software is occasioned by the way Cray economized his multiplier; about one-third of the bits that normal multiplier arrays generate have been left out of his multipliers, because they would contribute less than a unit to the last place of the final Cray-rounded product. Consequently, a Cray multiplier errs by almost a bit more than might have been expected. This error is compounded when division takes three multiplications to improve an approximate reciprocal of the divisor and then multiply the numerator by it. Square root compounds a few more multiplication errors.

The fast way drove out the slow, even though the fast was occasionally slightly wrong.

Making the World Safe for Floating Point, or Vice Versa

William Kahan was an undergraduate at the University of Toronto in 1953 when he learned to program its Ferranti-Manchester Mark-I computer. Because he entered the field early, Kahan became acquainted with a wide range of devices and a large proportion of the personalities active in computing; the numbers of both were small at that time. He has performed computations on slide rules, desktop

mechanical calculators, tabletop analog differential analyzers, and so on; he has used all but the earliest electronic computers and calculators mentioned in this book.

Kahan's desire to deliver reliable software led to an interest in error analysis that intensified during two years of postdoctoral study in England, where he became acquainted with Wilkinson. In 1960, he resumed teaching at Toronto, where an IBM 7090 had been acquired, and was granted free rein to tinker with its operating system, Fortran compiler, and runtime library. (He denies that he ever came near the 7090 hardware with a soldering iron but admits asking to do so.) One story from that time illuminates how misconceptions and numerical anomalies in computer systems can incur awesome hidden costs.

A graduate student in aeronautical engineering used the 7090 to simulate the wings he was designing for short takeoffs and landings. He knew such a wing would be difficult to control if its characteristics included an abrupt onset of stall, but he thought he could avoid that. His simulations were telling him otherwise. Just to be sure that roundoff was not interfering, he had repeated many of his calculations in double precision and gotten results much like those in single; his wings had stalled abruptly in both precisions. Disheartened, the student gave up.

Meanwhile Kahan replaced IBM's logarithm program (ALOG) with one of his own, which he hoped would provide better accuracy. While testing it, Kahan reran programs using the new version of ALOG. The student's results changed significantly; Kahan approached him to find out what had happened.

The student was puzzled. Much as the student preferred the results produced with the new ALOG—they predicted a gradual stall—he knew they must be wrong because they disagreed with his double precision results. The discrepancy between single and double precision results disappeared a few days later when a new release of IBM's double precision arithmetic software for the 7090 arrived. (The 7090 had no double precision hardware.) He went on to write a thesis about it and to build the wings; they performed as predicted. But that is not the end of the story.

In 1963, the 7090 was replaced by a faster 7094 with double precision floating-point hardware but with otherwise practically the same instruction set as the 7090. Only in double precision and only when using the new hardware did the wing stall abruptly again. A lot of time was spent to find out why. The 7094 hardware turned out, like the superseded 7090 software and the subsequent early S/360s, to lack a guard bit in double precision. Like so many programmers on those computers and on Cray's, the student discovered a trick to compensate for the lack of a guard digit; he wrote the expression $(0.5 - x) + 0.5$ in place of $1.0 - x$. Nowadays we would blush if we had to explain why such a trick might be necessary, but it solved the student's problem.

Meanwhile the lure of California was working on Kahan and his family; they came to Berkeley and he to the University of California. An opportunity presented itself in 1974 when accuracy questions induced Hewlett-Packard's calculator designers to call in a consultant. The consultant was Kahan, and his work

dramatically improved the accuracy of HP calculators, but that is another story. Fruitful collaboration with congenial coworkers, however, fortified him for the next and crucial opportunity.

It came in 1976, when John F. Palmer at Intel was empowered to specify the “best possible” floating-point arithmetic for all of Intel’s product line, as **Moore’s Law** made it now possible to create a whole floating-point unit on a single chip. The floating-point standard was originally started for the iAPX-432, but when it was late, Intel started the 8086 as a short-term emergency stand-in until the iAPX-432 was ready. The iAPX-432 never became popular, so the emergency stand-in became the standard-bearer for Intel. The 8087 floating-point coprocessor for the 8086 was contemplated. (A *coprocessor* is simply an additional chip that accelerates a portion of the work of a processor; in this case, it accelerated floating-point computation.)

Palmer had obtained his Ph.D. at Stanford a few years before and knew whom to call for counsel of perfection—Kahan. They put together a design that obviously would have been impossible only a few years earlier and looked not quite possible at the time. But a new Israeli team of Intel employees led by Rafi Navé felt challenged to prove their prowess to Americans and leaped at an opportunity to put something impossible on a chip—the 8087.

By now, floating-point arithmetics that had been merely diverse among mainframes had become chaotic among microprocessors, one of which might be host to a dozen varieties of arithmetic in ROM firmware or software. Robert G. Stewart, an engineer prominent in IEEE activities, got fed up with this anarchy and proposed that the IEEE draft a decent floating-point standard. Simultaneously, word leaked out in Silicon Valley that Intel was going to put on one chip some awesome floating point well beyond anything its competitors had in mind. The competition had to find a way to slow Intel down, so they formed a committee to do what Stewart requested.

Meetings of this committee began in late 1977 with a plethora of competing drafts from innumerable sources and dragged on into 1985, when IEEE Standard 754 for Binary Floating Point was made official. The winning draft was very close to one submitted by Kahan, his student Jerome T. Coonen, and Harold S. Stone, a professor visiting Berkeley at the time. Their draft was based on the Intel design, with Intel’s permission, of course, as simplified by Coonen. Their harmonious combination of features, almost none of them new, had at the outset attracted more support within the committee and from outside experts like Wilkinson than any other draft, but they had to win nearly unanimous support within the committee to win official IEEE endorsement, and that took time.

The First IEEE 754 Chips

In 1980, Intel became tired of waiting and released the 8087 for use in the IBM PC. The floating-point architecture of the companion 8087 had to be retrofitted into the 8086 opcode space, making it inconvenient to offer two operands per



instruction as found in the rest of the 8086. Hence the decision for one operand per instruction using a stack: “The designer’s task was to make a Virtue of this Necessity.” (Kahan’s [1990] history of the stack architecture selection for the 8087 is entertaining reading.)

Rather than the classical stack architecture, which has no provision for avoiding common subexpressions from being pushed and popped from memory into the top of the stack found in registers, Intel tried to combine a flat register file with a stack. The reasoning was that the restriction of the top of stack as one operand was not so bad since it only required the execution of an `FXCH` instruction (which swapped registers) to get the same result as a two-operand instruction, and `FXCH` was much faster than the floating-point operations of the 8087.

Since floating-point expressions are not that complex, Kahan reasoned that eight registers meant that the stack would rarely overflow. Hence, he urged that the 8087 use this hybrid scheme with the provision that stack overflow or stack underflow would interrupt the 8086 so that interrupt software could give the illusion to the compiler writer of an unlimited stack for floating-point data.

The Intel 8087 was implemented in Israel, and 7500 miles and 10 time zones made communication from California difficult. According to Palmer and Morse (*The 8087 Primer*, J. Wiley, New York, 1984, p. 93):

Unfortunately, nobody tried to write a software stack manager until after the 8087 was built, and by then it was too late; what was too complicated to perform in hardware turned out to be even worse in software. One thing found lacking is the ability to conveniently determine if an invalid operation is indeed due to a stack overflow.... Also lacking is the ability to restart the instruction that caused the stack overflow ...

The result is that the stack exceptions are too slow to handle in software. As Kahan [1990] says:

Consequently, almost all higher-level languages’ compilers emit inefficient code for the 80x87 family, degrading the chip’s performance by typically 50% with spurious stores and loads necessary simply to preclude stack over/under-flow....

I still regret that the 8087’s stack implementation was not quite so neat as my original intention.... If the original design had been realized, compilers today would use the 80x87 and its descendents more efficiently, and Intel’s competitors could more easily market faster but compatible 80x87 imitations.

In 1982, Motorola announced its 68881, which found a place in Sun 3s and Macintosh IIs; Apple had been a supporter of the proposal from the beginning. Another Berkeley graduate student, George S. Taylor, had soon designed a high-speed implementation of the proposed standard for an early superminicomputer (ELXSI 6400). The standard was becoming de facto before its final draft’s ink was dry.